第 20 章：Task 生命周期与并发结果收集

一句话先说结论：**coroutine 描述“要做什么”，Task 才是事件循环中可独立调度、可追踪结果、需要被明确回收的一次执行。**

真实服务经常要同时读取用户资料、搜索知识库、检查策略。写出三个 `async def` 并不难，难的是回答下面这些工程问题：

- 三项工作究竟是顺序执行，还是已经并发启动？
- 哪个对象保存每项工作的业务身份、结果和异常？
- 一个工具先失败时，其他工具会不会继续运行？
- 超时或提前返回后，尚未完成的工作由谁处理？
- 请求已经结束，后台任务还能否安全存活到应用退出？

本章会建立“创建、持有、等待、收集、回收”的完整心智模型。取消协议放在第 21 章，结构化并发 `TaskGroup` 放在第 22 章；这里先把现有项目中最常见的 `create_task()`、`gather()`、`as_completed()` 和 `wait()` 讲清楚。

学完后，你应该能够：

- 区分 coroutine、Task 和 Future；
- 判断一段代码何时真正获得并发性；
- 正确读取 Task 的成功、异常和取消状态；
- 根据结果顺序、提前返回和错误策略选择收集 API；
- 为每个 Task 找到明确的拥有者，并保证退出前没有遗留工作或未取得的异常。

20.1 先用 Java 心智模型找准位置

可以先把 `asyncio.Task` 类比成“提交给执行器后得到的 `Future` 句柄”，但不能完全等同。`asyncio` 默认不会为每个 Task 建立线程，也不会把工作自动丢进线程池。事件循环在一个线程内进行协作式调度：当前 Task 执行到真正会暂停的 `await` 时，其他 Task 才有机会前进。

Python 概念	可以先类比的 Java 概念	相似点	不能等同的地方
coroutine 对象	尚未提交的任务描述	都表示一项待执行工作	调用 `async def` 只创建对象，不会自

Python 概念	可以先类比的 Java 概念	相似点	不能等同的地方
coroutine 对象	尚未提交的任务描述	都表示一项待执行工作	调用 `async def` 只创建对象，不会自
`asyncio.Task[T]`	`Future<T>` / `CompletableFuture<T>` 加执行句柄	保存完成状态、结果或异常，可以被等待	Task 还负责推进 coroutine，并绑定到事件循环；不代表新线程
`asyncio.Future[T]`	`CompletableFuture<T>` 的结果占位能力	都能在未来变为成功、失败或取消	它是低层桥接对象，通常由事件循环或库 不负责执行 coroutin
`await task`	`Future.get()` 、`join()` 的等待意图	都在结果就绪后继续，并传播失败	`await` 暂停当前 Tas 不阻塞事件循环线程
`task.result()`	`Future.get()`	都尝试取得结果	Python 的 `result()` **不会等待**；未完成时 `InvalidStateError`
`asyncio.create_task(coro)`	`executor.submit(...)`	都产生一个可管理的执行句柄	`create_task()` 只把 coroutine 安排到当前事件循环 不提供线程级并行、持久化或重试

因此，“异步”不等于“多线程”，“并发推进”也不等于“CPU 并行”。如果 coroutine 内部直接调用阻塞式 HTTP 客户端或执行长时间 CPU 计算，事件循环仍会被卡住。如何隔离阻塞 I/O 和 CPU 工作属于后续章节；本章所有例子只讨论 Task 的调度与结果管理。

20.2 coroutine、Task、Future 是怎样连起来的

先看三者的数据流：

```
调用 async def
    |
    ▼
coroutine 对象：保存参数、局部状态和待执行位置
    |
```

README × readme × README × README × README × README × README × README
2502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

先看三者的数据流：

```
调用 async def
    |
    ▼
coroutine 对象：保存参数、局部状态和待执行位置
    |
    ├── 直接 await ──→ 由当前 Task 沿调用链推进
    |
    └── create_task(coro)
            |
            ▼
        新 Task：事件循环独立调度，并保存最终结果、异常或取消状态

Future：表示“将来会有最终状态”的低层 awaitable
Task 是 Future-like 对象，但还会主动推进它包装的 coroutine
```

调用 `async def` 只得到一次性的 coroutine 对象

```python
async def load_profile() -> dict[str, str]:
    return {"tier": "gold"}


profile_coro = load_profile()  # 函数体此时还没有运行
profile = await profile_coro  # 当前 Task 从这里开始推进它
```

coroutine 对象表示一次执行路径。它正常结束、失败或被取消后都不能倒带，再次 `await` 同一个对象会抛 `RuntimeError: cannot reuse already awaited coroutine`。

`create_task()` 增加了独立调度单元和生命周期句柄

```python
task = asyncio.create_task(load_profile(), name="tool:profile")
profile = await task
```

`create_task()` 把 coroutine 包装成 Task，安排它尽快执行，并立即返回句柄。调用方因此可以：

- 用 `await task` 等待并取得结果；
- 用 `done()` 、 `cancelled()` 检查状态；

`create_task()` 把 coroutine 包装成 Task，安排它尽快执行，并立即返回句柄。调用方因此可以：

- 用 `await task` 等待并取得结果；
- 用 `done()` 、 `cancelled()` 检查状态；
- 在完成后用 `result()` 、 `exception()` 读取最终状态；
- 用 `name=` 提供诊断名称，并用字典关联请求 ID、工具名等业务身份；
- 在不再需要结果时发出取消请求，并继续等待清理完成。

完成后的 Task 保存着最终状态，所以可以被多次 `await` 。它不会重新执行 coroutine；每个等待者观察到的都是同一次执行的结果或异常。

Future 主要用于衔接底层回调式 API

Future 只表示"结果以后才确定"。例如底层 I/O 可以在 socket 就绪时设置 Future 的结果，从而唤醒正在等待它的 Task。应用层通常不应手工调用 `Future()` 、 `set_result()` 或 `set_exception()` ；需要 Future 时，优先使用事件循环或库提供的对象。

20.3 并发性由"何时创建 Task"决定

假设两个工具没有数据依赖：

```
import asyncio


async def call_tool(name: str, delay: float) -> str:
    await asyncio.sleep(delay)  # 模拟不会阻塞事件循环的 I/O 等待
    return f"{name}:ok"
```

下面三种写法看起来接近，实际生命周期不同。

写法一：直接依次 `await` ，工作顺序执行

```
profile = await call_tool("profile", 0.2)
documents = await call_tool("search", 0.1)
```

第二个 `call_tool(...)` 表达式要等第一行完成后才会求值。因此第一项结束前，第二个 coroutine 对象都还不存在。后一步依赖前一步结果时，这正是正确写法。

写法二：创建一个就立刻等待，工作仍然不重叠

```
profile_task = asyncio.create_task(call_tool("profile", 0.2))
profile = await profile_task

search_task = asyncio.create_task(call_tool("search", 0.1))
documents = await search_task
```

这里确实创建了 Task，但第二个 Task 仍要等第一个完成后才创建。相比直接 `await`，它多了一个独立句柄，却没有让两项业务工作并发。

写法三：先创建全部 Task，再逐个等待，工作已经并发启动

```
profile_task = asyncio.create_task(call_tool("profile", 0.2))
search_task = asyncio.create_task(call_tool("search", 0.1))

profile = await profile_task
documents = await search_task
```

当前 Task 第一次暂停时，两个子 Task 都已经可以被事件循环推进。虽然读取结果时先写 `await profile_task`，搜索 Task 仍可更早完成；它会把结果保存在自身状态中，等调用方稍后读取。

所以不能只看 `await task1; await task2` 就判断串行。应先问：**两个 Task 是否已经在第一个 `await` 之前全部创建？**

不要把五种顺序混成一个“先后顺序”

顺序	由什么决定	是否有稳定契约
创建顺序	`create_task()` 语句的执行顺序	代码可直接确定
首次开始顺序	任务工厂和事件循环调度	不应依赖偶然顺序
完成顺序	I/O 就绪、工作量、异常和取消	通常不同于创建顺序
处理顺序	调用方选择等待哪个结果	由收集 API 和业务代码决定
聚合结果顺序	API 的返回契约	例如 `gather()` 明确按输入顺序

日志里“先创建”不等于“先完成”。测试并发语义时，应使用 `asyncio.Event` 这样的同步信号固定

README × | readme × | README × | README × | README × | README × | README × | README
72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

日志里"先创建"不等于"先完成"。测试并发语义时，应使用 `asyncio.Event` 这样的同步信号固定状态变化，不要用"睡 10 ms 的任务应该先完成"作为脆弱断言。

Python 3.14：`eager_start` 会改变开始时机

本项目目标版本是 Python 3.14，`create_task()` 可以显式传入 `eager_start`：

```
task = asyncio.create_task(cached_lookup(key), eager_start=True)
```

`eager_start=True` 允许 coroutine 在 `create_task()` 调用期间立即运行，直到第一次阻塞；如果它始终没有阻塞，Task 甚至可能在 `create_task()` 返回前就已经成功或失败。省略参数时，是否 eager 由事件循环的任务工厂决定。

这不只是性能开关，而是执行顺序的变化。任何依赖"`create_task()` 返回后我还能先修改共享状态"的代码，都可能在 eager 模式下失效。业务正确性不应依赖 Task 恰好在哪一轮循环首次运行。

20.4 Task 的最终状态应该怎样读取

Task 未完成时处于 pending。结束后只有三类最终结果：正常返回、抛出异常、完成取消。`done()` 对这三类都会返回 `True`，所以它不等于"执行成功"。

状态	`done()`	`cancelled()`	`result()`	`exception()`
pending	`False`	`False`	抛 `InvalidStateError`	抛 `InvalidStateError`
成功	`True`	`False`	返回结果	返回 `None`
普通异常	`True`	`False`	重新抛原异常	返回异常对象
完成取消	`True`	`True`	抛 `CancelledError`	抛 `CancelledError`

业务代码最常用也最安全的方式是直接等待：

```
try:
    value = await task
except ToolCallError as error:
    log_tool_failure(error)
```

`await task` 会先等待最终状态，再返回成功值或传播异常。只有在已经明确知道 Task 完成时，才适合

README × | readme × | README × | README × | README × | README × | README × | README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`await task` 会先等待最终状态，再返回成功值或传播异常。只有在已经明确知道 Task 完成时，才适合同步调用 `result()` 或 `exception()`，例如检查 `wait()` 返回的 `done` 集合。

状态检查顺序也很重要：取消同样属于 done，而 `exception()` 在取消状态会抛 `CancelledError`。通用检查通常先判断 `cancelled()`，再判断 `done()`，最后读取异常或结果。

`task.cancel()` 只是发出请求，不会同步把 Task 改成 cancelled。请求要在事件循环再次推进 Task 时送入 coroutine，coroutine 还有机会执行 `finally`。完整传播和资源清理规则见第 21 章。

20.5 创建 Task 就意味着承担所有权

直接 `await coroutine` 时，工作自然属于当前调用链。调用 `create_task()` 后，事件循环中出现了一个可独立存活的执行单元，创建者必须回答四个问题：

1. 谁保存它的强引用和业务身份？
2. 谁等待它完成？
3. 谁读取并处理它的结果或异常？
4. 所属请求或应用退出时，谁让它结束并完成清理？

在一次请求内，最直观的做法是显式保存所有 Task：

```
tasks = {
    "profile": asyncio.create_task(load_profile(), name="tool:profile"),
    "search": asyncio.create_task(search_documents(), name="tool:search"),
}

profile = await tasks["profile"]
documents = await tasks["search"]
```

Task 的 `name=` 主要服务于日志和诊断；`dict[str, Task]` 才明确保存了业务身份与强引用。

下面这种“提交后不管”不是可靠后台队列：

```
asyncio.create_task(report_metric())  # 教学反例
```

事件循环只保存 Task 的弱引用。没有其他强引用时，Task 可能在完成前被垃圾回收；即使它完成了，异常也可能无人取得，最终出现 `Task exception was never retrieved`。而且 `create_task()` 没有持久化、重试、跨进程恢复或应用退出协议。

README × readme × README × README × README × README × README × README
/f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

根据生命周期选择拥有者：

- 请求内的相关子任务：在请求返回前全部等待；新代码可在第 22 章改用 `TaskGroup` 。
- 跨请求但属于当前进程的任务：交给应用级监督器，登记名称，在 shutdown 时统一等待并记录结果。
- 进程重启后仍必须执行的工作：交给消息队列或持久化任务系统，不要依赖裸 Task。

20.6 `gather()`：等待一组结果，并保持输入顺序

`gather(*aws)` 接受多个 awaitable。裸 coroutine 会被自动安排为 Task；已经是 Task 的输入则保持原对象不变。全部成功时，它返回一个列表，**列表位置对应输入位置，而不是完成顺序。**

```
async def load(name: str, delay: float) -> str:
    await asyncio.sleep(delay)
    return name


tasks = [
    asyncio.create_task(load("profile", 0.3)),
    asyncio.create_task(load("search", 0.1)),
    asyncio.create_task(load("policy", 0.2)),
]
results = await asyncio.gather(*tasks)
assert results == ["profile", "search", "policy"]
```

无论三项工作的实际完成顺序怎样， `results` 都按 profile、search、policy 排列。因此 `gather()` 很适合“并发取得固定字段，最后按固定位置组装请求”的场景。

一个子 Task 失败时，兄弟 Task 默认继续运行

默认 `return_exceptions=False` 。第一个普通异常会立即传播给等待 `gather()` 的调用方，但其他输入不会因此被自动取消：

```
tasks = [asyncio.create_task(call()) for call in tool_calls]

try:
    results = await asyncio.gather(*tasks)
except Exception:
    # 原异常传播后，兄弟任务可能仍在运行；继续收齐所有最终状态。
    await asyncio.gather(*tasks, return_exceptions=True)
```

README | readme | README | README | README | README | README | README
7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
tasks = [asyncio.create_task(call()) for call in tool_calls]

try:
    results = await asyncio.gather(*tasks)
except Exception:
    # 原异常传播后，兄弟任务可能仍在运行；继续收齐所有最终状态。
    await asyncio.gather(*tasks, return_exceptions=True)
    raise
```

这段收尾代码选择“让兄弟任务自然完成”。如果业务要求“一项失败，其他项停止”，应按第 21 章的协议请求取消并等待清理，或在第 22 章使用具有结构化失败语义的 `TaskGroup` 。

还要区分 `gather()` 的聚合对象和子 Task：

- 子 Task 抛普通异常：聚合等待提前失败，其他子 Task 默认继续。
- `gather()` 自身被取消：取消会传给尚未完成的输入。
- 异常已传播、聚合对象已经 done 后，再调用 `gather.cancel()` ：不能倒回去取消仍在运行的兄弟。

如果异常直接离开顶层 coroutine， `asyncio.run()` 关闭事件循环时会取消遗留任务。此时日志里“兄弟被取消”是 runner 的退出清理，不能反推为 `gather()` 的普通异常策略。

`return_exceptions=True` 只改变交付形式

```
outcomes = await asyncio.gather(*tasks, return_exceptions=True)

for task, outcome in zip(tasks, outcomes, strict=True):
    if isinstance(outcome, BaseException):
        record_failure(task.get_name(), outcome)
    else:
        consume(outcome)
```

普通异常不再被 raise，而是作为对应输入位置的列表元素返回。它并不等于“错误已经处理”；调用方仍要记录、分类、转换或决定是否允许部分成功。

20.7 `as_completed()` ：谁先完成就先处理谁

当结果越早返回越有价值，例如更新 UI、输出流式进度或尝试多个只读数据源，可以使用 `as_completed()` 。本项目使用 Python 3.14，应优先采用异步迭代形式：

README × readme × README × README × README × README × README × README
nf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
tasks = {
    "profile": asyncio.create_task(load_profile(), name="tool:profile"),
    "search": asyncio.create_task(search_documents(), name="tool:search"),
}
task_names = {task: name for name, task in tasks.items()}

async for finished_task in asyncio.as_completed(tasks.values()):
    name = task_names[finished_task]
    result = await finished_task
    print(name, result)
```

从 Python 3.13 起，异步迭代时，如果输入本来就是 Task 或 Future，产出的就是对应原对象。因此可用对象身份找回业务名称。产出的 Task 已完成，但仍要 `await` 它或调用 `result()` 才能取得结果；如果该 Task 失败，异常会在这里传播。

旧代码还常使用普通迭代形式：

```
for next_result in asyncio.as_completed(tasks.values()):
    result = await next_result
```

普通 `for` 产出的是新的包装 coroutine，不是输入的原 Task，不能用它查询上面的 `task_names`。这两种形式都合法，但对象身份规则不同。

设置 `timeout=` 后，如果总超时时间内仍有输入未完成，迭代会抛 `TimeoutError`。`as_completed()` 不会自动取消剩余任务；调用方仍需保存它们，并决定继续等待、转交还是取消。

20.8 `wait()`：只报告 done/pending，策略由调用方决定

`wait()` 适合“先观察一部分状态，再决定下一步”：

```
done, pending = await asyncio.wait(
    tasks,
    timeout=1.0,
    return_when=asyncio.FIRST_COMPLETED,
)
```

Python 3.11 起不能把裸 coroutine 直接传给 `wait()`；应传入非空的 Task/Future iterable。它返回两个无序集合，不保留输入位置语义。

README × | readme × | README × | README × | README × | README × | README × | README
72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

return_when	返回时机
FIRST_COMPLETED	任意一项成功、失败或取消完成
FIRST_EXCEPTION	任意一项以异常结束；若始终无异常，则等同 ALL_COMPLETED
ALL_COMPLETED	全部完成或取消；默认值

timeout 到期时， wait() **不会抛 TimeoutError，也不会取消输入。**未完成项只会出现在 pending 集合中。因为成功、异常和取消都属于 done，读取结果时仍要分类：

```
for task in done:
    if task.cancelled():
        report_cancelled(task)
    elif (error := task.exception()) is not None:
        report_failed(task, error)
    else:
        consume(task.result())

# pending 仍在运行，不能直接丢弃。
if pending:
    await asyncio.gather(*pending, return_exceptions=True)
```

示例最后选择继续等待 pending。真实业务也可以把它们转交给另一个明确拥有者，或者请求取消并等待取消完成；无论选哪种， wait() 本身都不会替调用方完成生命周期管理。

20.9 怎样选择结果收集方式

需求	推荐方式	关键契约
后一步依赖前一步结果	直接顺序 await	依赖关系明确，不需要并发
全部成功后按固定字段组装结果	显式创建 Task，再 gather()	并发等待，结果按输入顺序
允许部分失败，并要收齐每项状态	gather(..., return_exceptions=True)	异常成为对应位置的元素，仍需处理
谁先完成就先消费谁	异步迭代 as_completed()	按完成顺序产出，显式 Task 身份可保留
等首项，首个异常或一段时间后再决策	wait()	返回 done/pending，不自动处置

等首项、首个异常或一段时间后再决策	`wait()`	返回 done/pending，不自动处置 pending
一组相关子任务需要统一失败和退出边界	第 22 章的 `TaskGroup`	结构化并发提供更强生命周期保证
进程级长期后台工作	应用监督器或持久化任务系统	需要明确所有权、停止、异常和恢复策略

`gather()` 、 `as_completed()` 和 `wait()` 都能"同时等待多个对象"，但它们解决的是不同问题：

- `gather()` 关注最终结果列表；
- `as_completed()` 关注完成事件的先后；
- `wait()` 关注某个时点的状态分区。

先写清业务需要哪一种结果语义，再选 API，不要机械地把它们互换。

20.10 Java 工程师最容易踩的坑

常见误判	正确理解
调用 `async def` 后工作已经开始	只创建了 coroutine 对象；必须 await、创建 Task 或交给其他调度 API
`create_task()` 会启动一个新线程	默认只是在当前事件循环增加协作调度单元
`await task1; await task2` 一定串行	若 Task 已提前全部创建，它们已经能够并发推进
创建顺序就是完成顺序	完成取决于 I/O、工作量和失败；聚合 API 还有自己的排列规则
`task.result()` 像 `Future.get()` 一样阻塞	pending 时立即抛 `InvalidStateError` ；等待要用 `await`
`done()` 为真就一定有成功结果	异常和取消也属于 done
Task 和 coroutine 都只能 await 一次	coroutine 对象只能执行一次；完成 Task 可以重复 await
`gather()` 一项失败会自动取消全部	默认普通异常只向等待者传播，兄弟继续运行
`return_exceptions=True` 已经处理了异常	它只改变异常的交付形式
`as_completed()` 两种迭代形式返回同类对象	异步迭代可产出原 Task；普通迭代产出包装 coroutine

README | readme | README | README | README | README | README | README
/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
2.50
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`wait()` 返回 pending 说明它们已取消	pending 仍在运行或等待，必须继续管理
`create_task()` 可以充当可靠任务队列	它没有持久化、重试、恢复和天然监督者

20.11 动手练习

先写出预期状态和事件顺序，再运行代码验证。不要用精确耗时作为正确性依据。·

练习 1：比较三种启动方式

为 `load_profile` 、 `search_docs` 、 `check_policy` 准备 started/release/finished 三组 Event，分别实现：

1. 直接逐个 `await` ；
2. 创建一个 Task 后立刻 `await` ；
3. 先创建全部 Task，再逐个 `await` 。

让控制器在第三种方式中按 policy、profile、search 放行。验证前两种方式中后一个工具只会在前一个完成后开始；第三种方式要先观察到三者都 started，并区分完成顺序与读取结果顺序。

练习 2：处理 `gather()` 的部分失败

显式创建三个有名字的 Task，让 search 在另外两项仍被 Event 阻塞时抛 `LookupError("index unavailable")` 。捕获默认 `gather()` 的异常后验证：

- search 已 done；
- 两个兄弟仍 pending 且没有取消；
- 放行兄弟后，用 `return_exceptions=True` 收齐三项最终状态；
- 程序退出时没有未取得的异常或遗留 Task。

练习 3：完成顺序与状态快照

先用 Python 3.14 的 `async for ... in as_completed(...)` 按 search、policy、profile 的完成顺序处理三个显式 Task，并用 `dict[Task, str]` 找回身份。再创建一组 Task，验证：

- `wait(timeout=0)` 对未放行的 Task 返回空 done 和完整 pending；
- `FIRST_COMPLETED` 只报告第一项最终状态；
- `wait()` 没有取消 pending；
- 离开入口前，调用方明确处理了所有 pending。

README × | README × | readme × | README × | README × | README × | README × | README
wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

练习 4：为后台任务补上拥有者

审查下面的反例，分别说明强引用、业务身份、异常收集和退出清理缺失在哪里：

```
async def handle_request() -> str:
    asyncio.create_task(write_audit_log())
    asyncio.create_task(report_metric())
    return "accepted"
```

实现一个最小应用级监督器：注册时保存 Task 和名称；`shutdown()` 时给有限任务发送结束信号，等待全部最终状态，逐项记录成功或异常，最后再删除引用。

20.12 运行本章示例

本章只使用 Python 标准库，不需要安装额外依赖。

文件	作用
`README.md`	建立 Task 生命周期、状态与结果收集的心智模型
`example.py`	用 `asyncio.Event` 门控九组本地实验，并用断言验证 API 契约

从仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/20_asyncio_tasks/example.py
```

`-B` 只是不生成 `.pyc`，不改变异步行为。运行时重点观察：

- 直接 `await`、立即等待新 Task、先创建全部 Task 的开始与完成顺序；
- `eager_start=True` 如何改变 `create_task()` 返回前后的事件；
- pending、成功、异常、取消四种状态下的查询结果；
- `gather()` 的完成顺序、结果顺序和异常传播后兄弟 Task 的状态；
- `as_completed()` 的处理顺序，以及 `wait()` 的 done/pending；
- 入口结束前是否还有未回收的其他 Task。

项目已配置 Ruff 和 Pyright，可继续检查：

```
.venv/bin/ruff check java_to_python/chapters/20_asyncio_tasks
.venv/bin/ruff format --check java_to_python/chapters/20_asyncio_tasks
```

README | readme | README | README | README | README | README | README
/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-710bdd9cd540585deaef81028528.html
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`-B` 只是不生成 `.pyc`，不改变异步行为。运行时重点观察：

- 直接 `await` 、立即等待新 Task、先创建全部 Task 的开始与完成顺序；
- `eager_start=True` 如何改变 `create_task()` 返回前后的事件；
- pending、成功、异常、取消四种状态下的查询结果；
- `gather()` 的完成顺序、结果顺序和异常传播后兄弟 Task 的状态；
- `as_completed()` 的处理顺序，以及 `wait()` 的 done/pending；
- 入口结束前是否还有未回收的其他 Task。

项目已配置 Ruff 和 Pyright，可继续检查：

```
.venv/bin/ruff check java_to_python/chapters/20_asyncio_tasks
.venv/bin/ruff format --check java_to_python/chapters/20_asyncio_tasks
.venv/bin/pyright java_to_python/chapters/20_asyncio_tasks/example.py
```

`java_to_python/run_all.py` 目前只运行前 10 章，因此第 20 章需要使用上面的独立命令。

20.13 本章小结

- coroutine 是一次性的待执行路径；Task 是被事件循环调度并保存最终状态的执行句柄。
- `create_task()` 会增加一个必须被拥有、等待、收集和清理的生命周期对象。
- 是否并发主要看 Task 何时创建，而不是后来逐个 `await` 的书写顺序。
- `done()` 包含成功、异常和取消；pending 时调用 `result()` 或 `exception()` 不会等待。
- `gather()` 按输入顺序返回；普通异常默认不会取消兄弟 Task。
- `as_completed()` 按完成顺序交付；`wait()` 只返回 done/pending 状态快照。
- 任何提前返回都不能自动解除调用方对剩余 Task 的责任。
- 裸 `create_task()` 不是后台队列。没有强引用、异常处理和退出协议，就没有可靠的任务所有权。

建议合上文档后回答三个问题：两个 Task 先创建再依次 `await` 为什么仍是并发？`gather()` 传播异常后兄弟 Task 由谁收尾？`wait()` 返回 pending 后，哪个对象仍对它们负责？如果能结合 Task 状态和所有权回答清楚，本章的核心就掌握了。

参考资料

- Python 3.14：Coroutines and Tasks
- Python 3.14：Futures

```python
"""第 20 章：Task 生命周期与并发结果收集。

示例使用 asyncio.Event 控制三个本地工具替身的开始与完成条件，不访问真实网络。
断言检查事件和对象状态，不把精确耗时或偶然的调度先后当作结论。
"""

# 推荐按文件从上往下读：先认识工具调用的数据模型和 Event 控制器，再看九组实验。
# 每组实验都会先创建工具计划与控制信号，随后启动 Task，由控制器按确定顺序打开 release，
# 最后用结果、Task 状态和事件列表做断言。这样观察到的是 asyncio 的契约，而非机器快慢。
# asyncio 提供事件循环、Task 和并发收集工具；dataclass 用来定义轻量且可比较的数据对象；
# pairwise() 则在第一个实验中成对检查“前一个完成后，后一个才开始”。
import asyncio
from dataclasses import dataclass, field
from itertools import pairwise


# 这是纯同步的展示辅助函数。title: str 和 -> None 是类型提示，便于阅读器和静态检查器理解
# 参数与返回值；Python 运行时并不会仅凭这些注解自动拒绝其他类型。
def print_section(title: str) -> None:
    # f-string 会先计算花括号里的 title，再把结果嵌入字符串；开头的 \n 先输出一个空行。
    print(f"\n=== {title} ===")


# events 是由调用方持有的可变 list。append() 会原地修改同一个列表，所以实验结束后仍可
# 从调用方看到完整轨迹；这类似把同一个 Java List 引用传入方法，而不是复制一份列表。
def record(events: list[str], message: str) -> None:
    """同时记录并打印事件，让调度过程既可观察又可断言。"""
    events.append(message)
    print(f"  事件 | {message}")


# 自定义异常继承 RuntimeError，让失败工具走一条可识别的业务异常路径；这里定义类并不会抛异常。
class ToolCallError(RuntimeError):
    """本地工具替身的可预期失败。"""


```

```python

# @dataclass 会根据字段生成 __init__、__repr__ 和按值比较等样板方法，可类比精简的 Java
# record/POJO；但 Python 类型提示依然不是编译期强制约束。frozen=True 禁止正常的字段再赋值，
# 让一份调用计划在多个 Task 间共享时保持不变。
@dataclass(frozen=True)
class ToolPlan:
    # name 是日志与映射键，value 是成功返回的数据；should_fail 控制是否走演示失败分支。
    name: str
    value: str
    # 默认值只在调用方未传此参数时使用，因此前两个计划会成功，显式传 True 的计划会失败。
    should_fail: bool = False


# 工具成功后不直接返回裸字符串，而是把“哪个工具”和“返回值”一起保存，便于后续验证身份。
@dataclass(frozen=True)
class ToolResult:
    tool_name: str
    value: str


# 这组 dataclass 是每次工具调用的控制面。它没有 frozen=True，因为实验会改变三个 Event
# 内部的开关状态；字段本身仍始终引用各自的 Event 对象。
@dataclass
class ToolControl:
    """started/release/finished 分别控制并观察一个工具的生命周期。"""

    # default_factory 接收的是可调用对象 asyncio.Event，而不是已经创建好的 Event。
    # 每次构造 ToolControl 都会分别调用它三次，因此不同字段、不同工具不会误共享同一信号。
    # Event 初始为关闭状态；wait() 会暂停，set() 会打开并唤醒等待者，且 set() 本身不需 await。
    started: asyncio.Event = field(default_factory=asyncio.Event)
    release: asyncio.Event = field(default_factory=asyncio.Event)
    finished: asyncio.Event = field(default_factory=asyncio.Event)


# 圆括号里的逗号创建 tuple。tuple 本身不可变，适合把三份固定计划作为所有实验的共同输入；
# 元素 ToolPlan 也因 frozen=True 不允许普通字段赋值。
```

```python
# 圆括号里的逗号创建 tuple。tuple 本身不可变，适合把三份固定计划作为所有实验的共同输入；
# 元素 ToolPlan 也因 frozen=True 不允许普通字段赋值。
TOOLS: tuple[ToolPlan, ToolPlan, ToolPlan] = (
    ToolPlan("资料工具", "用户等级=gold"),
    ToolPlan("搜索工具", "找到 2 条文档"),
    ToolPlan("策略工具", "允许生成摘要"),
)


# 输入是若干 ToolPlan 组成的 tuple；返回字典把每个唯一工具名映射到一套全新的控制信号。
def make_controls(plans: tuple[ToolPlan, ...]) -> dict[str, ToolControl]:
    # 这是字典推导式：for 每取出一个 plan，就计算键 plan.name 和值 ToolControl()。
    # `...` 表示 tuple 可包含任意数量的 ToolPlan，不是函数体中的省略占位符。
    return {plan.name: ToolControl() for plan in plans}


# async def 定义 coroutine 函数；调用 controlled_tool(...) 只创建 coroutine 对象，函数体要到
# 被直接 await 或包装进 Task 后才会推进。返回注解表示正常路径最终产生 ToolResult。
async def controlled_tool(
    plan: ToolPlan,
    control: ToolControl,
    events: list[str],
    completion_order: list[str],
) -> ToolResult:
    """等待 release 事件后成功或失败，用于替代不可控的真实 I/O。"""
    # current_task() 查询当前正在推进这段 coroutine 的 Task。直接 await 时它就是上层调用者
    # Task；create_task 后则是新建的业务 Task。条件表达式处理理论上的 None，避免直接取名报错。
    current: Task[Any] | None = asyncio.current_task()
    task_name: str = current.get_name() if current is not None else "无 Task"
    record(events, f"{plan.name}: 开始（运行于 {task_name}）")
    # 先打开 started，控制器才知道本工具已经真正进入函数体，而不只是创建了 coroutine/Task。
    control.started.set()

    try:
        # Event.wait() 返回一个 awaitable。release 尚未 set 时，await 在此交还事件循环控制权；
        # 控制器随后可推进并决定何时放行。这代表"外部结果尚未就绪"，却不依赖真实等待时长。
```

```
 88  async def controlled_tool(
103      try:
104          # Event.wait() 返回一个 awaitable。release 尚未 set 时，await 在此交还事件循环控制权；
105          # 控制器随后可推进并决定何时放行。这代表“外部结果尚未就绪”，却不依赖真实等待时长。
106          await control.release.wait()
107          # 放行后根据计划选择失败或成功。失败也先写 completion_order，记录本工具结束工作的顺序。
108          if plan.should_fail:
109              completion_order.append(plan.name)
110              record(events, f"{plan.name}: 失败")
111              raise ToolCallError(f"{plan.name} 调用失败")
112
113          # 成功路径构造新的不可变结果对象，记录完成顺序和可读事件，再由 return 交给等待者。
114          result = ToolResult(plan.name, plan.value)
115          completion_order.append(plan.name)
116          record(events, f"{plan.name}: 完成 -> {plan.value}")
117          return result
118      except asyncio.CancelledError:
119          # cancel() 的请求在下一个可取消的 await 处以 CancelledError 送进 coroutine。
120          # 这里只观察取消已经到达；bare raise 会重新抛当前异常，让 Task 正确进入 cancelled 状态，
121          # 而不是把取消误吞成一次正常返回。完整的取消与清理协议留到第 21 章。
122          completion_order.append(plan.name)
123          record(events, f"{plan.name}: 收到取消")
124          raise
125      finally:
126          # finally 在成功、普通异常、取消三条路径都会执行。finished 因而表示“已离开工具调用”，
127          # 控制器可安全推进下一个步骤；它不表示一定成功。
128          control.finished.set()
129
130
131  # 等待所有工具都报告 started。函数本身不关心这些工具最终按什么顺序完成。
132  async def wait_until_all_started(controls: dict[str, ToolControl]) -> None:
133      """不猜首次调度顺序，明确等到所有工具都已进入等待状态。"""
134      # values() 取出各 ToolControl；生成器表达式逐个创建 Event.wait() coroutine。
135      # 前缀 * 把它们展开为 gather 的多个位置参数。gather 并发等待，全部完成后这一行才返回。
136      await asyncio.gather(*(control.started.wait() for control in controls.values()))
137
138
```

```python
# 这个控制器配合“每次只启动一个工具”的实验：看到当前工具开始，就放行并等它完全结束，
# 然后 for 才进入下一轮。plans 的排列因此直接决定完成顺序。
async def drive_releases_after_each_start(
    plans: tuple[ToolPlan, ...],
    controls: dict[str, ToolControl],
    events: list[str],
) -> None:
    """逐个放行顺序调用；这个 Task 模拟三个彼此独立的外部服务。"""
    for plan in plans:
        # 用 plan.name 找到与本轮工具配套的三个 Event；字典下标在缺键时会抛 KeyError。
        control: ToolControl = controls[plan.name]
        # 即使控制器比工具先运行也没关系：它会先停在 started.wait()，不会过早放行。
        await control.started.wait()
        record(events, f"控制器：放行 {plan.name}")
        control.release.set()
        # 等到工具的 finally 打开 finished，才允许处理下一个计划。
        await control.finished.wait()


# 这个控制器用于并发实验：先建立“全部已开始”的屏障，再按 order 指定的业务名逐个放行。
async def drive_release_order(
    controls: dict[str, ToolControl],
    order: tuple[str, ...],
    events: list[str],
) -> None:
    """先确认全部工具开始，再按指定次序制造确定的完成顺序。"""
    await wait_until_all_started(controls)
    record(events, "控制器：已观察到全部工具开始")
    for name in order:
        record(events, f"控制器：放行 {name}")
        controls[name].release.set()
        # 每次等待对应 finished 后再放行下一项，使 completion_order 与 order 稳定一致。
        await controls[name].finished.wait()


# 把“为每份计划创建有名字的 Task，并保存所有权”集中到一个同步辅助函数中。
```

```
173
174 # 把"为每份计划创建有名字的 Task，并保存所有权"集中到一个同步辅助函数中。
175 # 参数列表中的单独 * 表示其后的 label 只能按关键字传入，调用点必须写 label="..."。
176 def create_tool_tasks(
177     plans: tuple[ToolPlan, ...],
178     controls: dict[str, ToolControl],
179     events: list[str],
180     completion_order: list[str],
181     *,
182     label: str,
183 ) -> dict[str, asyncio.Task[ToolResult]]:
184     """创建并返回有名字的 Task；调用方必须继续持有并最终回收它们。"""
185     # 类型注解说明：键是工具名，值是最终产出 ToolResult 的 Task。局部字典会通过 return
186     # 转交给调用方；调用方继续保存它，也就继续持有所有 Task 的强引用与业务身份。
187     tasks: dict[str, asyncio.Task[ToolResult]] = {}
188     for plan in plans:
189         record(events, f"创建 Task：{plan.name}")
190         # controlled_tool(...) 先产生尚未执行的 coroutine，再由 create_task 包装并安排运行。
191         # name 只用于诊断和日志，不影响调度优先级。基础实验不传 eager_start，并在入口断言
192         # 当前使用默认任务工厂，所以这次同步调用返回时 coroutine 函数体尚未开始。
193         tasks[plan.name] = asyncio.create_task(
194             controlled_tool(plan, controls[plan.name], events, completion_order),
195             name=f"{label}:{plan.name}",
196         )
197     return tasks
198
199
200 # 第一组实验不为业务工具新建 Task，而是在入口 Task 的调用链中依次推进三个 coroutine。
201 async def demonstrate_direct_await() -> None:
202     print_section("1. 直接 await：三个工具严格顺序调用")
203     # 这两个列表分别保留细粒度事件和完成工具名。空列表字面量每执行一次都会创建新对象，
204     # 各实验不会共享记录；list[str] 仍是静态类型提示。
205     events: list[str] = []
206     completion_order: list[str] = []
207     controls: dict[str, ToolControl] = make_controls(TOOLS)
208     # 控制器必须作为独立 Task 运行，否则当前调用链停在工具的 release.wait() 后，无人放行。
200     controller: Task["str"] = asyncio.create_task(
```

```python
# 第一组实验不为业务工具新建 Task，而是在入口 Task 的调用链中依次推进三个 coroutine。
async def demonstrate_direct_await() -> None:
    print_section("1. 直接 await：三个工具严格顺序调用")
    # 这两个列表分别保留细粒度事件和完成工具名。空列表字面量每执行一次都会创建新对象，
    # 各实验不会共享记录；list[str] 仍是静态类型提示。
    events: list[str] = []
    completion_order: list[str] = []
    controls: dict[str, ToolControl] = make_controls(TOOLS)
    # 控制器必须作为独立 Task 运行，否则当前调用链停在工具的 release.wait() 后，无人放行。
    controller: Task[None] = asyncio.create_task(
        drive_releases_after_each_start(TOOLS, controls, events),
        name="direct-await:service-controller",
    )

    results: list[ToolResult] = []
    for plan in TOOLS:
        # 没有为工具创建 Task：controlled_tool 在当前入口 Task 的调用链里运行到 return。
        # 当前循环要等 result 到手后才会进入下一轮，所以后一个 coroutine 此刻甚至尚未创建。
        result: ToolResult = await controlled_tool(
            plan,
            controls[plan.name],
            events,
            completion_order,
        )
        results.append(result)
    # 三个工具返回后，控制器通常也已走完；仍显式 await，确保它的最终状态与异常被回收。
    await controller

    # 列表推导式保持 TOOLS 的迭代顺序，得到预期名列表；下一行再用相同写法读取实际结果。
    expected_names: list[str] = [plan.name for plan in TOOLS]
    assert [result.tool_name for result in results] == expected_names
    assert completion_order == expected_names
    # pairwise(TOOLS) 依次产出相邻二元组：(资料, 搜索)、(搜索, 策略)。
    for previous, following in pairwise(TOOLS):
        # index() 找前一个工具“完成”事件的位置；next() 从生成器取第一个匹配的“开始”位置。
        # enumerate(events) 每轮给出 (下标, 消息)，for 中的逗号把二元组解包到两个名字。
```

```
201  async def demonstrate_direct_await() -> None:
233      for previous, following in pairwise(TOOLS):
234          # index() 找前一个工具“完成”事件的位置；next() 从生成器取第一个匹配的“开始”位置。
235          # enumerate(events) 每轮给出（下标，消息），for 中的逗号把二元组解包到两个名字。
236          previous_finished: int = events.index(f"{previous.name}：完成 -> {previous.value}")
237          following_started: int = next(
238              index
239              for index, message in enumerate(events)
240              if message.startswith(f"{following.name}：开始")
241          )
242          # 比较下标而非耗时：每一对都满足完成事件在开始事件之前，便证明业务调用严格顺序。
243          assert previous_finished < following_started
244      print("  结论：后一个工具只在前一个工具返回后才开始")
245
246
247  # 第二组把两种都使用 create_task 的写法放在一起，证明“是否并发”取决于创建与暂停的时机，
248  # 不能只看代码里有没有 create_task，也不能只看后面是否逐个 await。
249  async def demonstrate_creation_patterns() -> None:
250      print_section("2. create_task 后立刻 await，与先全部创建再 await")
251
252      # 第一小组仍使用顺序控制器。每创建一个业务 Task，就立刻停下来等待它完成。
253      immediate_events: list[str] = []
254      immediate_completion: list[str] = []
255      immediate_controls: dict[str, ToolControl] = make_controls(TOOLS)
256      immediate_controller: Task[None] = asyncio.create_task(
257          drive_releases_after_each_start(TOOLS, immediate_controls, immediate_events),
258          name="immediate-await:service-controller",
259      )
260
261      # 字典既保留每个 Task 的强引用，也保留工具名到 Task 的身份关系。
262      immediate_tasks: dict[str, asyncio.Task[ToolResult]] = {}
263      for plan in TOOLS:
264          # 这里确实新建了业务 Task，它会由事件循环独立推进，而非沿当前调用链直接推进。
265          task: Task[ToolResult] = asyncio.create_task(
266              controlled_tool(
267                  plan,
```

```
async def demonstrate_creation_patterns() -> None:

    # 字典既保留每个 Task 的强引用，也保留工具名到 Task 的身份关系。
    immediate_tasks: dict[str, asyncio.Task[ToolResult]] = {}
    for plan in TOOLS:
        # 这里确实新建了业务 Task，它会由事件循环独立推进，而非沿当前调用链直接推进。
        task: Task[ToolResult] = asyncio.create_task(
            controlled_tool(
                plan,
                immediate_controls[plan.name],
                immediate_events,
                immediate_completion,
            ),
            name=f"immediate-await:{plan.name}",
        )
        immediate_tasks[plan.name] = task
        # 当前入口 Task 在此暂停并等待新 Task 的最终状态；这一步完成前，for 不会开始下一轮，
        # 因此下一个业务 Task 尚未创建，三个工具的工作仍然没有重叠。
        await task
    # 同样明确回收控制器，而不是因“看起来早已完成”就丢弃引用。
    await immediate_controller

    expected_names: list[str] = [plan.name for plan in TOOLS]
    assert immediate_completion == expected_names
    # str.join(iterable) 用箭头连接所有名称，不会改变原列表。
    print("  创建一个就立刻 await: ", " -> ".join(immediate_completion))

    # 第二小组先在同步函数 create_tool_tasks 内完成三次创建，中间没有任何 await。
    concurrent_events: list[str] = []
    concurrent_completion: list[str] = []
    concurrent_controls: dict[str, ToolControl] = make_controls(TOOLS)
    concurrent_tasks: dict[str, Task[ToolResult]] = create_tool_tasks(
        TOOLS,
        concurrent_controls,
        concurrent_events,
        concurrent_completion,
        label="create-all",
```

```python
async def demonstrate_creation_patterns() -> None:
    # any(...) 只要找到一个真值就返回 True。默认任务工厂不会在 create_task 调用内部立刻推进
    # coroutine，所以此刻记录里只有“创建 Task”，不应出现工具函数体写下的“开始”。
    assert not any(": 开始" in message for message in concurrent_events)

    # 该 tuple 与 TOOLS 的输入顺序故意不同，用来区分创建、完成和结果处理顺序。
    forced_completion = ("策略工具", "资料工具", "搜索工具")
    # create_task 返回后控制器与三个业务 Task 都处于可调度状态；入口在稍后的 await 才真正让出控制。
    concurrent_controller: Task[None] = asyncio.create_task(
        drive_release_order(
            concurrent_controls,
            forced_completion,
            concurrent_events,
        ),
        name="create-all:service-controller",
    )

    awaited_results: list[str] = []
    # 虽然这里按 TOOLS 顺序逐个 await，但三个 Task 早已全部安排运行，所以这不是顺序启动。
    # 入口先等“资料工具”时，控制器仍能让策略工具先完成；完成的 Task 会保存结果，等未来读取。
    for plan in TOOLS:
        result: ToolResult = await concurrent_tasks[plan.name]
        awaited_results.append(result.tool_name)
    await concurrent_controller

    assert concurrent_completion == list(forced_completion)
    # awaited_results 记录的是 for 选择等待/读取的顺序，因此仍与输入 expected_names 相同。
    assert awaited_results == expected_names
    # 先定位控制器写入的“全部开始”屏障，再寻找任意第一个成功完成事件。
    all_started_marker: int = concurrent_events.index("控制器：已观察到全部工具开始")
    first_completion: int = next(
        index for index, message in enumerate(concurrent_events) if ": 完成 ->" in message
    )
    # 这个断言排除了“某工具还没开始，另一个就已完成”的情况，不依赖三者内部的开始先后。
    assert all_started_marker < first_completion

```

```
249  async def demonstrate_creation_patterns() -> None:
332
333      print("  实际完成顺序: ", " -> ".join(concurrent_completion))
334      print("  逐个 await 得到结果的顺序: ", " -> ".join(awaited_results))
335      print("  结论: await task1 再 await task2, 不会撤销它们已经获得的并发性")
336
337
338  # 这个极小 coroutine 专门探测"函数体何时开始"。label 既作为日志内容，也作为最终返回值。
339  async def eager_probe(label: str, events: list[str]) -> str:
340      record(events, f"{label}: 函数体开始")
341      # sleep(0) 不等待真实时长，却仍是协作检查点：当前 Task 主动暂停，让事件循环推进其他工作。
342      # eager 模式只会同步推进到这个首次阻塞点；实验没有基于耗时做断言。
343      await asyncio.sleep(0)
344      record(events, f"{label}: 恢复并完成")
345      return label
346
347
348  # 第三组是 Python 3.14 专题：只改变 create_task 的 eager_start 参数，比较返回瞬间的事件列表。
349  async def demonstrate_python314_eager_start() -> None:
350      print_section("3. Python 3.14 补充: eager_start 会改变开始时机")
351      # get_running_loop() 只能在正在运行的事件循环中调用。任务工厂为 None 表示使用默认工厂，
352      # 避免外部安装的 eager 工厂改变基础实验的开始语义。
353      loop: AbstractEventLoop = asyncio.get_running_loop()
354      assert loop.get_task_factory() is None
355
356      normal_events: list[str] = []
357      # 默认工厂下，普通 create_task 只安排稍后运行；同步调用返回时 eager_probe 尚未写事件。
358      normal: Task[str] = asyncio.create_task(eager_probe("普通 Task", normal_events), name="normal-probe")
359      print("  普通 create_task 返回时的事件: ", normal_events)
360      assert normal_events == []
361      # await 让出控制并回收 Task，之后普通探针会完整执行。
362      await normal
363
364      eager_events: list[str] = []
365      # eager_start=True 允许 coroutine 在 create_task 调用内部立即运行，直到首次阻塞。
366      eager: Task[str] = asyncio.create_task(
367          eager_probe("eager Task", eager_events),
```

```python
async def demonstrate_python314_eager_start() -> None:

    eager_events: list[str] = []
    # eager_start=True 允许 coroutine 在 create_task 调用内部立即运行，直到首次阻塞。
    eager: Task[str] = asyncio.create_task(
        eager_probe("eager Task", eager_events),
        name="eager-probe",
        eager_start=True,
    )
    # 因此"函数体开始"已经发生，但 sleep(0) 后的恢复尚未发生，Task 也还没有最终状态。
    print("  eager create_task 返回时的事件：", eager_events)
    assert eager_events == ["eager Task：函数体开始"]
    assert not eager.done()
    await eager

    print("  结论：创建顺序不能自动等同于开始顺序；eager 模式尤其会改变时机")


# 调用此 async 函数会得到一次性的原生 coroutine 对象，第四组用它和可复用最终状态的 Task 对照。
async def one_shot_coroutine() -> str:
    return "原生 coroutine 的结果"


# 第四组依次制造 pending、成功、失败和取消状态，并在各状态下调用 Task 查询方法。
async def demonstrate_task_state() -> None:
    print_section("4. Task 状态、结果、异常与重复 await")
    events: list[str] = []
    completion_order: list[str] = []
    plan = ToolPlan("状态工具", "ready")
    control = ToolControl()
    task: Task[ToolResult] = asyncio.create_task(
        controlled_tool(plan, control, events, completion_order),
        name="state:success",
    )
    # 等到 started 后，工具确定已经运行并阻塞在仍关闭的 release 上，因此 Task 稳定处于 pending。
    await control.started.wait()

```

```python
async def demonstrate_task_state() -> None:

    print("  未完成: done=", task.done(), "cancelled=", task.cancelled())
    # 每个二元组保存"展示名称"和一个已绑定方法。for 会将二元组解包到 method_name/accessor;
    # 直接调用 accessor() 等价于分别调用 task.result()、task.exception(), 不会等待 Task。
    for method_name, accessor in (("result", task.result), ("exception", task.exception)):
        try:
            accessor()
        except asyncio.InvalidStateError as error:
            # pending Task 没有最终值或异常, 两种读取器都会立刻抛 InvalidStateError。
            print(f"  未完成时调用 {method_name}():", type(error).__name__)
        else:
            # else 只在 try 中没有异常时执行; 到达这里说明 asyncio 行为与实验预期不符。
            raise AssertionError(f"未完成 Task 的 {method_name}() 应抛 InvalidStateError")

    # 打开 release 后, await 真正等待最终状态并取得 coroutine 的 return 值。
    control.release.set()
    first: ToolResult = await task
    # Task 保存了最终状态, 因此可再次 await; 它不会重新执行 controlled_tool, 也不会重复事件。
    second: ToolResult = await task
    print("  完成: done=", task.done(), "cancelled=", task.cancelled())
    print("  两次 await 同一 Task: ", first, "|", second)
    print("  result(): ", task.result(), "exception(): ", task.exception())
    # is 比较对象身份而不是值相等: 两次 await 返回的正是同一个 ToolResult 实例。
    assert first is second
    assert task.exception() is None

    # 再建一份 should_fail=True 的计划, 隔离演示 Task 如何保存普通异常。
    failed_plan = ToolPlan("失败状态工具", "不会返回", should_fail=True)
    failed_control = ToolControl()
    failed: Task[ToolResult] = asyncio.create_task(
        controlled_tool(failed_plan, failed_control, events, completion_order),
        name="state:failure",
    )
    await failed_control.started.wait()
    failed_control.release.set()
    try:
```

```
386  async def demonstrate_task_state() -> None:
432      failed_control.release.set()
433      try:
434          # await 失败 Task 会在等待点把 ToolCallError 重新抛给当前调用方。
435          await failed
436      except ToolCallError as error:
437          print("  await 失败 Task:", type(error).__name__, str(error))
438      else:
439          raise AssertionError("失败 Task 应向等待者重新抛出工具异常")
440
441      # Task 已完成且未取消，此时 exception() 不再抛 InvalidStateError，而是返回保存的异常对象。
442      stored_error: BaseException | None = failed.exception()
443      assert isinstance(stored_error, ToolCallError)
444      try:
445          # result() 只在成功状态返回值；失败状态会重新抛出 Task 保存的原异常。
446          failed.result()
447      except ToolCallError:
448          print("  失败后 exception() 返回异常对象, result() 重新抛出它")
449      else:
450          raise AssertionError("失败 Task 的 result() 应重新抛出原异常")
451
452      # 这里没有 create_task：one_shot 直接引用一次 coroutine 调用产生的执行对象。
453      one_shot: CoroutineType[Any, Any, str] = one_shot_coroutine()
454      await one_shot
455      try:
456          # 原生 coroutine 不是最终状态容器；一次执行结束后不能倒带或再次推进。
457          await one_shot
458      except RuntimeError as error:
459          print("  原生 coroutine 第二次 await:", type(error).__name__, str(error))
460      else:
461          raise AssertionError("已完成的原生 coroutine 不可复用")
462
463      # 最后一种状态只作取消预告：Task 已开始且正停在可取消的 Event.wait() 上。
464      cancel_plan = ToolPlan("取消预告工具", "不会返回")
465      cancel_control = ToolControl()
466      cancelled_task: Task[ToolResult] = asyncio.create_task(
467          controlled_tool(cancel_plan, cancel_control, events, completion_order)
```

```
386  async def demonstrate_task_state() -> None:
465      cancel_control = ToolControl()
466      cancelled_task: Task[ToolResult] = asyncio.create_task(
467          controlled_tool(cancel_plan, cancel_control, events, completion_order),
468          name="state:cancel-preview",
469      )
470      await cancel_control.started.wait()
471      # cancel(message) 发送取消请求并返回请求是否被接受；它不是同步把 cancelled 标志设为 True。
472      accepted: bool = cancelled_task.cancel("第 21 章再讲完整协议")
473      print("  cancel() 请求已接受: ", accepted, "立即 cancelled=", cancelled_task.cancelled())
474      try:
475          # 必须 await，才能让 CancelledError 送入 coroutine，执行其 except/finally，并回收最终状态。
476          await cancelled_task
477      except asyncio.CancelledError:
478          print("  取消送达并回收后: cancelled=", cancelled_task.cancelled())
479      else:
480          raise AssertionError("本例中的 Task 应进入取消完成状态")
481      # and 具有短路语义；这里只在 done() 为真后再检查 cancelled()，验证最终确为取消完成。
482      assert cancelled_task.done() and cancelled_task.cancelled()
483
484
485  # 第五组让完成顺序故意偏离输入顺序，观察 gather 按哪一种顺序排列返回列表。
486  async def demonstrate_gather_order() -> None:
487      print_section("5. gather: 并发等待, 结果保持输入顺序")
488      events: list[str] = []
489      completion_order: list[str] = []
490      controls: dict[str, ToolControl] = make_controls(TOOLS)
491      tasks: dict[str, Task[ToolResult]] = create_tool_tasks(
492          TOOLS,
493          controls,
494          events,
495          completion_order,
496          label="gather-order",
497      )
498      # 列表推导式按 TOOLS 的固定顺序，从字典中取出三个原 Task；这就是稍后 gather 的输入顺序。
499      ordered_tasks: list[Task[ToolResult]] = [tasks[plan.name] for plan in TOOLS]
```

```python
async def demonstrate_gather_order() -> None:
    )
    # 列表推导式按 TOOLS 的固定顺序，从字典中取出三个原 Task；这就是稍后 gather 的输入顺序。
    ordered_tasks: list[Task[ToolResult]] = [tasks[plan.name] for plan in TOOLS]
    # 完成顺序被明确固定为“搜索、策略、资料”，与 ordered_tasks 的“资料、搜索、策略”不同。
    forced_completion = ("搜索工具", "策略工具", "资料工具")
    controller: Task[None] = asyncio.create_task(
        drive_release_order(controls, forced_completion, events),
        name="gather-order:service-controller",
    )

    # * 在调用位置把列表拆成三个位置参数。因输入已经是 Task，gather 会观察这些原对象，
    # 并返回一个独立的聚合 Future-like 对象；此刻尚未 await，所以先有机会检查它的状态。
    aggregate: Future[list[ToolResult]] = asyncio.gather(*ordered_tasks)
    waiter: Task[Any] | None = asyncio.current_task()
    # current_task() 理论上可返回 None，条件表达式在取 get_name() 前先保护这一情况。
    print("  等待者 Task:", waiter.get_name() if waiter else None)
    # `in` 按相等性检查容器成员；Task 使用对象身份语义，这里证明聚合对象不是任一输入 Task。
    print("  gather 聚合对象是独立 Future-like:", aggregate not in ordered_tasks)
    print("  await 前 aggregate.done():", aggregate.done())
    # await aggregate 会等待所有输入成功结束，并给出按输入位置排列的 ToolResult 列表。
    results: list[ToolResult] = await aggregate
    await controller
                                I
    input_order: list[str] = [plan.name for plan in TOOLS]
    result_order: list[str] = [result.tool_name for result in results]
    # 第一条断言验证真实完成顺序，第二条验证 gather 的结果顺序；两者故意不同。
    assert completion_order == list(forced_completion)
    assert result_order == input_order
    print("  完成顺序:", " -> ".join(completion_order))
    print("  gather 结果顺序:", " -> ".join(result_order))


# 第六组的控制器先只放行失败项，然后等待调用方确认“已经观察兄弟状态”，再放行其余项。
# caught_by_waiter 是双方的同步信号，使 pending 结论不受事件循环碰巧多调度几步的影响。
async def drive_gather_failure(
    plans: tuple[ToolPlan, ...],
```

```python

# 第六组的控制器先只放行失败项，然后等待调用方确认“已经观察兄弟状态”，再放行其余项。
# caught_by_waiter 是双方的同步信号，使 pending 结论不受事件循环碰巧多调度几步的影响。
async def drive_gather_failure(
    plans: tuple[ToolPlan, ...],
    controls: dict[str, ToolControl],
    caught_by_waiter: asyncio.Event,
    events: list[str],
) -> None:
    """先让失败发生；确认等待者已捕获后，才放行两个兄弟 Task。"""
    await wait_until_all_started(controls)
    # next() 取得生成器产生的第一个失败计划名；本实验数据确保恰好存在一个失败计划。
    failed_name: str = next(plan.name for plan in plans if plan.should_fail)
    controls[failed_name].release.set()
    # finished 在工具的 finally 中打开，所以返回后失败异常已经写入对应 Task 的最终状态。
    await controls[failed_name].finished.wait()

    # 这道门确保“捕获异常时兄弟仍 pending”不是调度速度造成的偶然现象：在调用方显式
    # set() 之前，控制器不可能打开兄弟 Task 的 release。
    await caught_by_waiter.wait()
    for plan in plans:
        # continue 立即跳到下一轮，失败项不会被重复放行；其余两个才进入下面的清理步骤。
        if plan.name == failed_name:
            continue
        controls[plan.name].release.set()
        await controls[plan.name].finished.wait()


# 把 Task 的内部组合状态转换成适合打印的标签。检查顺序很重要：取消也属于 done，
# 所以先排除 cancelled，再判断 pending，最后才可安全调用 exception()。
def state_label(task: asyncio.Task[ToolResult]) -> str:
    if task.cancelled():
        return "cancelled"
    if not task.done():
        return "pending"
    return "failed" if task.exception() is not None else "succeeded"

```

```
565
566 # 默认 gather 会把第一个普通异常传播给等待者，但不会因此取消其他输入 Task。
567 async def demonstrate_gather_failure() -> None:
568     print_section("6. gather 异常：传播一个异常，不自动取消兄弟 Task")
569     # 只有"搜索工具"启用失败开关；三份 ToolControl 仍各自独立。
570     plans: tuple[ToolPlan, ToolPlan, ToolPlan] = (
571         ToolPlan("资料工具", "用户等级=gold"),
572         ToolPlan("搜索工具", "不会返回", should_fail=True),
573         ToolPlan("策略工具", "允许生成摘要"),
574     )
575     events: list[str] = []
576     completion_order: list[str] = []
577     controls: dict[str, ToolControl] = make_controls(plans)
578     tasks: dict[str, Task[ToolResult]] = create_tool_tasks(
579         plans,
580         controls,
581         events,
582         completion_order,
583         label="gather-failure",
584     )
585     # 保存输入顺序和原 Task，既便于 gather 聚合，也便于异常传播后继续管理兄弟 Task。
586     ordered_tasks: list[Task[ToolResult]] = [tasks[plan.name] for plan in plans]
587     caught_by_waiter = asyncio.Event()
588     controller: Task[None] = asyncio.create_task(
589         drive_gather_failure(plans, controls, caught_by_waiter, events),
590         name="gather-failure:service-controller",
591     )
592
593     # 没传 return_exceptions，默认值 False 表示普通异常通过 await aggregate 直接抛给等待者。
594     aggregate: Future[list[ToolResult]] = asyncio.gather(*ordered_tasks)
595     try:
596         await aggregate
597     except ToolCallError as error:
598         # 控制器尚停在 caught_by_waiter：搜索 Task 已失败，两个兄弟还停在各自 release.wait()。
599         print("  gather 向等待者传播：", type(error).__name__, str(error))
600         print("  aggregate.done()：", aggregate.done())
601         for plan in plans:
```

```python
async def demonstrate_gather_failure() -> None:
        print("  aggregate.done()=", aggregate.done())
        for plan in plans:
            print(f"  捕获时 {plan.name}: {state_label(tasks[plan.name])}")
        # 这些断言把“聚合 Future 已因异常完成”与“兄弟 Task 仍未完成且未取消”区分开。
        assert tasks["搜索工具"].done()
        assert not tasks["资料工具"].done()
        assert not tasks["策略工具"].done()
        assert not tasks["资料工具"].cancelled()
        assert not tasks["策略工具"].cancelled()
        # 状态观察完才通知控制器放行兄弟，确保上面的证据稳定可复现。
        caught_by_waiter.set()
    else:
        raise AssertionError("默认 gather 应传播第一个工具异常")

    # 仍在同一个事件循环中，把相同的原 Task 再交给 gather。完成 Task 可以重复等待；
    # return_exceptions=True 让 ToolCallError 作为对应位置的列表元素返回，而不是再次中断等待，
    # 因而两个兄弟有机会继续完成，所有 Task 的结果/异常也得到统一回收。
    collected: list[ToolResult | BaseException] = await asyncio.gather(*ordered_tasks, return_exceptions=True)
    await controller
    # 索引 1 对应 ordered_tasks 的第二项“搜索工具”；gather 的结果按输入位置而非完成顺序排列。
    assert isinstance(collected[1], ToolCallError)
    # all(...) 要求每个 Task 都完成；any(...) 只要发现一个已取消就为真，外层 not 要求一个也没有。
    assert all(task.done() for task in ordered_tasks)
    assert not any(task.cancelled() for task in ordered_tasks)
    # 字典推导式保留“工具名 -> 最终标签”，items() 每轮把键和值解包给 name/task。
    print("  再收集后的状态: ", {name: state_label(task) for name, task in tasks.items()})
    print("  return_exceptions=True 的输入位置 2: ", repr(collected[1]))
    print("  完成顺序: ", " -> ".join(completion_order))


# 第七组不要求固定字段顺序，而是哪个 Task 先完成就立即处理哪个，同时仍要找回其业务名称。
async def demonstrate_as_completed() -> None:
    print_section("7. as_completed: 按完成顺序处理, 并保留 Task 身份")
    events: list[str] = []
    completion_order: list[str] = []
    controls: dict[str, ToolControl] = make_controls(TOOLS)
```

```python
# 第七组不要求固定字段顺序，而是哪个 Task 先完成就立即处理哪个，同时仍要找回其业务名称。
async def demonstrate_as_completed() -> None:
    print_section("7. as_completed: 按完成顺序处理, 并保留 Task 身份")
    events: list[str] = []
    completion_order: list[str] = []
    controls: dict[str, ToolControl] = make_controls(TOOLS)
    tasks: dict[str, Task[ToolResult]] = create_tool_tasks(
        TOOLS,
        controls,
        events,
        completion_order,
        label="as-completed",
    )
    # tasks 原来是“名称 -> Task”；这里用字典推导式反转成“Task -> 名称”。Task 对象可哈希，
    # 所以能作为 dict 键；后面无需依赖输入下标或解析 Task 的诊断名称来识别业务身份。
    task_names: dict[Task[ToolResult], str] = {task: name for name, task in tasks.items()}
    forced_completion = ("策略工具", "搜索工具", "资料工具")
    controller: Task[None] = asyncio.create_task(
        drive_release_order(controls, forced_completion, events),
        name="as-completed:service-controller",
    )

    processed: list[str] = []
    # as_completed 返回可异步迭代对象；async for 会在每个下一项完成前协作等待，而不会阻塞线程。
    # Python 3.14 使用这种异步迭代形式时，传入的原 Task 会按完成顺序原样产出，适合身份映射。
    # 这不同于兼容旧版的普通 `for ... in as_completed(...)`：后者产出包装 coroutine。
    async for finished_task in asyncio.as_completed(tasks.values()):
        assert isinstance(finished_task, asyncio.Task), "3.14 异步迭代应产出原 Task"
        # 只有原对象身份得到保留，finished_task 才能直接命中反向字典。
        tool_name: str = task_names[finished_task]
        # 产出的 Task 已完成，await 会立刻交付已保存结果；若它失败，则会在这里抛出异常。
        result: ToolResult = await finished_task
        assert result.tool_name == tool_name
        processed.append(tool_name)
        print(f"  立即处理 {tool_name}: {result.value}")
    await controller
```

```python
async def demonstrate_as_completed() -> None:

    # 控制器逐个放行并等待 finished，所以异步迭代的处理顺序应与确定的完成顺序完全一致。
    assert processed == list(forced_completion)
    assert completion_order == processed
    print("  处理顺序: ", " -> ".join(processed))


# 第八组使用 wait 取得 done/pending 两个集合。wait 只报告状态，所有权和后续处理仍归调用方。
async def demonstrate_wait() -> None:
    print_section("8. wait: done/pending、超时和返回条件")
    events: list[str] = []
    completion_order: list[str] = []
    controls: dict[str, ToolControl] = make_controls(TOOLS)
    tasks: dict[str, Task[ToolResult]] = create_tool_tasks(
        TOOLS,
        controls,
        events,
        completion_order,
        label="wait-first-completed",
    )
    # set(...) 保存无顺序、元素唯一的 Task 集合；wait 的返回值也是集合，不承诺输入位置语义。
    task_set: set[Task[ToolResult]] = set(tasks.values())
    task_names: dict[Task[ToolResult], str] = {task: name for name, task in tasks.items()}
    # 先确认所有业务 Task 已停在 release 上，后面的 timeout=0 才会稳定得到全 pending。
    await wait_until_all_started(controls)

    # timeout=0 表示立刻查看一次状态。wait 超时不会抛 TimeoutError，也不会取消未完成 Task。
    timeout_done: set[Task[ToolResult]], timeout_pending: set[Task[ToolResul… = await asyncio.wait(task_set, timeout
    print("  timeout=0: done=", len(timeout_done), "pending=", len(timeout_pending))
    # set() 创建空集合；不能写 {}，因为空的 {} 是 dict。
    assert timeout_done == set()
    assert timeout_pending == task_set
    assert not any(task.cancelled() for task in timeout_pending)

    # 只放行策略工具。FIRST_COMPLETED 会在任一 Task 成功、失败或取消完成时提前返回。
    controls["策略工具"].release.set()
```

```python
async def demonstrate_wait() -> None:

    # 只放行策略工具。FIRST_COMPLETED 会在任一 Task 成功、失败或取消完成时提前返回。
    controls["策略工具"].release.set()
    done: set[Task[ToolResult]], pending: set[Task[ToolResul… = await asyncio.wait(task_set, return_when=asyncio.FIR
    # 两个集合没有固定迭代顺序，转成名称集合后再与期望集合比较。
    done_names: set[str] = {task_names[task] for task in done}
    pending_names: set[str] = {task_names[task] for task in pending}
    print("  FIRST_COMPLETED done: ", done_names)
    print("  FIRST_COMPLETED pending: ", pending_names)
    assert done_names == {"策略工具"}
    assert pending_names == {"资料工具", "搜索工具"}
    assert not any(task.cancelled() for task in pending)

    # wait 只报告 pending；拥有者仍需决定如何结束并回收它们。这里按反向映射找到控制信号，
    # 打开剩余 release，随后 gather 等待原 task_set 中的全部 Task，避免实验留下游离工作。
    for task in pending:
        controls[task_names[task]].release.set()
    await asyncio.gather(*task_set)
    assert all(task.done() for task in task_set)

    # 第二小组专门演示 FIRST_EXCEPTION：只有搜索工具抛普通异常，两个兄弟仍保持 pending。
    failure_plans: tuple[ToolPlan, ToolPlan, ToolPlan] = (
        ToolPlan("资料工具", "用户等级=gold"),
        ToolPlan("搜索工具", "不会返回", should_fail=True),
        ToolPlan("策略工具", "允许生成摘要"),
    )
    failure_events: list[str] = []
    failure_order: list[str] = []
    failure_controls: dict[str, ToolControl] = make_controls(failure_plans)
    failure_tasks: dict[str, Task[ToolResult]] = create_tool_tasks(
        failure_plans,
        failure_controls,
        failure_events,
        failure_order,
        label="wait-first-exception",
    )
```

```python
async def demonstrate_wait() -> None:
    failure_set: set[Task[ToolResult]] = set(failure_tasks.values())
    failure_names: dict[Task[ToolResult], str] = {task: name for name, task in failure_tasks.items()}
    await wait_until_all_started(failure_controls)
    # 只放行预定失败项，确保 FIRST_EXCEPTION 的唤醒原因明确。
    failure_controls["搜索工具"].release.set()

    failed_done: set[Task[ToolResult]], failed_pending: set[Task[ToolResul… = await asyncio.wait(
        failure_set,
        return_when=asyncio.FIRST_EXCEPTION,
    )
    print("  FIRST_EXCEPTION done:", {failure_names[task] for task in failed_done})
    print("  FIRST_EXCEPTION pending:", {failure_names[task] for task in failed_pending})
    assert {failure_names[task] for task in failed_done} == {"搜索工具"}
    # 本例确定 done 集合只有一个元素。iter(...) 创建迭代器，next(...) 取出该失败 Task；
    # Task 已完成且未取消，所以现在调用 exception() 是安全的。
    assert isinstance(next(iter(failed_done)).exception(), ToolCallError)
    assert not any(task.cancelled() for task in failed_pending)

    # 提前返回没有替调用方处理 pending；这里明确让兄弟完成，再把失败异常作为值收集。
    for task in failed_pending:
        failure_controls[failure_names[task]].release.set()
    outcomes: list[ToolResult | BaseException] = await asyncio.gather(*failure_set, return_exceptions=True)
    # failure_set 是无序集合，因此不检查异常的固定下标，只要求所有结果中存在 ToolCallError。
    assert any(isinstance(outcome, ToolCallError) for outcome in outcomes)
    assert all(task.done() for task in failure_set)


# 第九组模拟超出当前请求调用栈、但仍属于应用生命周期的后台任务；重点是所有权而非并发 API。
async def demonstrate_background_ownership() -> None:
    print_section("9. 后台 Task 强引用、异常收集与退出回收")
    plans: tuple[ToolPlan, ToolPlan] = (
        ToolPlan("审计写入", "audit-ok"),
        ToolPlan("指标上报", "不会返回", should_fail=True),
    )
    events: list[str] = []
```

```python
# 第九组模拟超出当前请求调用栈、但仍属于应用生命周期的后台任务；重点是所有权而非并发 API。
async def demonstrate_background_ownership() -> None:
    print_section("9. 后台 Task：强引用、异常收集与退出回收")
    plans: tuple[ToolPlan, ToolPlan] = (
        ToolPlan("审计写入", "audit-ok"),
        ToolPlan("指标上报", "不会返回", should_fail=True),
    )
    events: list[str] = []
    completion_order: list[str] = []
    controls: dict[str, ToolControl] = make_controls(plans)

    # 这个 dict 是明确的拥有者：键对 Task 保存强引用，值保存业务身份。若只写
    # asyncio.create_task(...) 而丢掉返回值，就缺少后续等待、异常收集和退出清理的句柄。
    owned_tasks: dict[asyncio.Task[ToolResult], str] = {}
    for plan in plans:
        task: Task[ToolResult] = asyncio.create_task(
            controlled_tool(plan, controls[plan.name], events, completion_order),
            name=f"background:{plan.name}",
        )
        owned_tasks[task] = plan.name

    # 等全部后台 Task 确实开始后，字典仍保存两项；len() 观察的是拥有者中的引用数。
    await wait_until_all_started(controls)
    print("  运行中的已拥有 Task 数：", len(owned_tasks))

    # 模拟应用退出信号：values() 取出每个控制器并打开 release，让这些有限任务可以结束。
    for control in controls.values():
        control.release.set()
    # list(owned_tasks) 遍历 dict 时得到键，因此快照是 Task 列表。快照冻结本轮等待对象，
    # 后面处理结果期间即使拥有者发生变化，也不会破坏 Task 与 outcomes 的位置对应关系。
    task_snapshot: list[Task[ToolResult]] = list(owned_tasks)
    # return_exceptions=True 保证一个后台 Task 失败时，仍等待其他 Task 并把每项最终状态收齐。
    outcomes: list[ToolResult | BaseException] = await asyncio.gather(*task_snapshot, return_exceptions=True)

    # zip 按位置把 Task 与 gather 结果配对；strict=True 要求两个序列等长，长度不符就抛 ValueError，
    # 防止静默遗漏结果。for 再把每个二元组解包为 task 和 outcome。
```

```python
async def demonstrate_background_ownership() -> None:
    # zip 按位置把 Task 与 gather 结果配对；strict=True 要求两个序列等长，长度不符就抛 ValueError，
    # 防止静默遗漏结果。for 再把每个二元组解包为 task 和 outcome。
    for task, outcome in zip(task_snapshot, outcomes, strict=True):
        owner_name: str = owned_tasks[task]
        # CancelledError 也属于 BaseException；用它分类可覆盖成功值之外的异常/取消结果。
        if isinstance(outcome, BaseException):
            print(f"  已收集 {owner_name} 的异常：{type(outcome).__name__}: {outcome}")
        else:
            print(f"  已收集 {owner_name} 的结果：{outcome.value}")

    # 只有结果和异常都处理完，才 clear() 原地清空字典，释放拥有者保存的强引用。
    owned_tasks.clear()
    # 空容器在布尔上下文中为 False，所以 not owned_tasks 验证拥有者已经清空。
    assert not owned_tasks


# 这是九组异步实验的总入口。它本身会被 asyncio.run 包装为顶层 Task，并按书写顺序逐组等待。
async def run_examples() -> None:
    current: Task[Any] | None = asyncio.current_task()
    # current 理论上可为 None；本函数实际由 asyncio.run 驱动，因此通常能把顶层 Task 改成可读名称。
    if current is not None:
        current.set_name("chapter20-main")

    # 基础实验依赖默认的非 eager 任务工厂，所以在入口尽早断言环境没有被外部代码改写。
    loop: AbstractEventLoop = asyncio.get_running_loop()
    print("Python 3.14 基础实验任务工厂：", loop.get_task_factory())
    assert loop.get_task_factory() is None

    # 每个 await 都等本组实验完整回收内部 Task 后才进入下一组，避免事件和状态相互干扰。
    await demonstrate_direct_await()
    await demonstrate_creation_patterns()
    await demonstrate_python314_eager_start()
    await demonstrate_task_state()
    await demonstrate_gather_order()
    await demonstrate_gather_failure()
```

```
811
812 # 这是九组异步实验的总入口。它本身会被 asyncio.run 包装为顶层 Task，并按书写顺序逐组等待。
813 async def run_examples() -> None:
814     current: Task[Any] | None = asyncio.current_task()
815     # current 理论上可为 None；本函数实际由 asyncio.run 驱动，因此通常能把顶层 Task 改成可读名称。
816     if current is not None:
817         current.set_name("chapter20-main")
818
819     # 基础实验依赖默认的非 eager 任务工厂，所以在入口尽早断言环境没有被外部代码改写。
820     loop: AbstractEventLoop = asyncio.get_running_loop()
821     print("Python 3.14 基础实验任务工厂：", loop.get_task_factory())
822     assert loop.get_task_factory() is None
823
824     # 每个 await 都等本组实验完整回收内部 Task 后才进入下一组，避免事件和状态相互干扰。
825     await demonstrate_direct_await()
826     await demonstrate_creation_patterns()
827     await demonstrate_python314_eager_start()
828     await demonstrate_task_state()
829     await demonstrate_gather_order()
830     await demonstrate_gather_failure()
831     await demonstrate_as_completed()
832     await demonstrate_wait()
833     await demonstrate_background_ownership()
834
835     # all_tasks() 返回当前事件循环中尚未完成的 Task 集合，其中也包含正在执行本行的入口 Task。
836     # 集合推导式排除当前 Task 和已完成项；每个辅助 Task 都应在所属实验内被等待，最终应为空。
837     unfinished: set[Task[Any]] = {
838         task
839         for task in asyncio.all_tasks()
840         if task is not asyncio.current_task() and not task.done()
841     }
842     print("\n入口结束前未回收的其他 Task 数：", len(unfinished))
843     assert not unfinished
844     print("所有第 20 章示例与运行时断言均已完成。")
845
846
847 # 普通同步 main 是程序与 asyncio 的边界，asyncio.run 会创建事件循环、运行 run_examples
```

```python
async def run_examples() -> None:
    unfinished: set[Task[Any]] = {
        task
        for task in asyncio.all_tasks()
        if task is not asyncio.current_task() and not task.done()
    }
    print("\n入口结束前未回收的其他 Task 数: ", len(unfinished))
    assert not unfinished
    print("所有第 20 章示例与运行时断言均已完成。")


# 普通同步 main 是程序与 asyncio 的边界。asyncio.run 会创建事件循环，运行 run_examples
# 到完成，并在退出时关闭循环；一个进程里通常只在最外层调用它，不能嵌套在已运行的循环中。
def main() -> None:
    asyncio.run(run_examples())


# Python 直接执行本文件时，模块名是 "__main__"，因此调用 main；若被其他模块 import，
# 解释器只完成上面的定义，不自动启动九组实验。这相当于显式声明脚本入口。
if __name__ == "__main__":
    main()

```